

```

    }
  }
  .alignItems(HorizontalAlign.Start);
  // 帖子网格
  if (this.posts.length === 0 && this.loading) {
    Column() {
      LoadingProgress().width(28).height(28);
      Text('加载中...').fontSize(13).fontColor($r('app.color.text_secondary')).
margin({ top: 10 });
    }
    .layoutWeight(1).width('100%').justifyContent(FlexAlign.Center);
  } else if (this.posts.length === 0) {
    Column() {
      Text(this.error ? ('加载失败: ' + this.error) : 'TA 还没有发布过帖子')
        .fontSize(14).fontColor($r('app.color.text_secondary'));
    }
    .layoutWeight(1).width('100%').justifyContent(FlexAlign.Center);
  } else {
    Refresh({ refreshing: this.isRefreshing, builder: this.refreshBuilder
  }) {
    Grid() {
      ForEach(this.posts, (post: Post) => {
        GridItem() {
          PostCard({ post: post })
            .onClick(() => {
              this.goDetail(post);
            });
        }, (post: Post) => post.id !== undefined ? post.id.toString() : ''
      );
      if (this.finished) {
        GridItem() {
          Text('没有更多了')
            .fontSize(12).fontColor($r('app.color.text_secondary'))
            .width('100%').textAlign(TextAlign.Center).padding(16);
        }
        .columnStart(0).columnEnd(1);
      }
    }
    .columnsTemplate('1fr 1fr')
    .columnsGap(12).rowsGap(12).padding(12)
    .onScrollIndex((_start: number, end: number) => {
      if (end >= this.posts.length - 1) {
        this.loadMore();
      }
    })
    .onRefreshing(() => {
      this.isRefreshing = true;
      this.refresh().finally(() => {
        this.isRefreshing = false;
      });
    })
    .layoutWeight(1).width('100%');
  }
}
}
}
.width('100%').padding({ top: this.statusBarHeight }).height('100%').backgro
undColor($r('app.color.bg_primary'))
.transition(buildPageTransition())
}
}
}

```

```
// ===== ./entry/src/main/ets/services/api.ets =====
import { http } from '@kit.NetworkKit';
import { Post, Comment, Tag, User, Pagination, Genre, Provider, BindingItem, BindAccountBody, UnbindResult, NotificationItem, UnreadCount, UserProfile, FollowUser, FollowListResult, ReportBody, PendingPost, ModerateAction, NotificationPrefs, BlockUser, PrivacySettingsData, UserExportData, DebateVoteResult, Conversation, DmMessage } from '../models/types';
// 本地后端地址 (按运行环境替换):
// - 模拟器/真机本地开发: 用 hdc 反向端口转发, 连 127.0.0.1:3000 即可, 无需关心电脑 IP (IP 变化)
// 设置命令: hdc rport tcp:3000 tcp:3000 (模拟器每次重启后重跑一次即可)
// - 真机/同局域网直连: 'http://<你电脑局域网IP>:3000'
// - 上架后: 'https://api.bigbluebook.com'
export const BASE_URL = 'http://127.0.0.1:3000';
// token 由 utils/auth.ts 统一管理 (PersistentStorage 持久化), request 直接从 AppStorage 获取
export interface ApiResult<T> {
  code: number;
  data: T;
  message: string;
}
// 判断错误是否为网络连接类失败 (后端未启动 / hdc rport 未建立 / 超时等)
function isNetworkConnectionError(message: string): boolean {
  const m: string = message.toLowerCase();
  return m.includes('connect') || m.includes('refused') || m.includes('econnrefused') ||
    m.includes('unreachable') || m.includes('timed out') || m.includes('network');
};
async function request<T>(method: http.RequestMethod, path: string, body?: Object): Promise<T> {
  const url = BASE_URL + path;
  const req = http.createHttp();
  const token: string = (AppStorage.get<string>('authToken') ?? '') as string;
  const header: Record<string, string> = { 'Content-Type': 'application/json' };
  if (token) {
    header['Authorization'] = 'Bearer ' + token;
  }
  try {
    const resp = await req.request(url, {
      method,
      header,
      extraData: body ? JSON.stringify(body) : undefined,
      expectDataType: http.HttpDataType.OBJECT,
      connectTimeout: 10000,
      readTimeout: 10000,
    });
    // 先尝试解析 body 中的业务错误 (后端 400/409/404 等返回 {code, message})
    // 让后端业务错误 message 透传到前端 catch 块 (如"内容含敏感词"、"你已举报过该内容")
    const result = resp.result as ApiResult<T>;
    if (result && typeof result.code === 'number') {
      if (result.code !== 0) {
        throw new Error(result.message || '请求失败');
      }
      return result.data;
    }
    // fallback: 非 JSON 响应或无 code 字段
    if (resp.responseCode !== 200) {
      throw new Error('HTTP ' + resp.responseCode);
    }
    throw new Error('请求失败');
  } catch (e) {
    const err: Error = e as Error;
  }
}
```

```

const rawMsg: string = err.message ? err.message : '请求发生未知错误';
// 连接失败 (ECONNREFUSED 等) 给出可操作提示, 避免直接透传系统底层错误
if (isNetworkConnectionError(rawMsg)) {
  throw new Error('无法连接后端服务, 请确认后端已启动且 hdc rport 已建立 (模拟器重启后需)');
}
throw new Error(rawMsg);
} finally {
  req.destroy();
}
}

interface ApiClient {
  get: <T>(path: string) => Promise<T>;
  post: <T>(path: string, body?: Object) => Promise<T>;
  put: <T>(path: string, body?: Object) => Promise<T>;
  del: <T>(path: string) => Promise<T>;
}

export const api: ApiClient = {
  get: <T>(path: string): Promise<T> => request<T>(http.RequestMethod.GET, path)
  , post: <T>(path: string, body?: Object): Promise<T> => request<T>(http.RequestMethod.POST, path, body),
  put: <T>(path: string, body?: Object): Promise<T> => request<T>(http.RequestMethod.PUT, path, body),
  del: <T>(path: string): Promise<T> => request<T>(http.RequestMethod.DELETE, path),
};

// ===== 领域类型 =====
export type SortType = 'hot' | 'latest' | 'recommend';
export interface ListResult<T> {
  list: T[];
  pagination: Pagination;
}

// 详情接口返回的帖子自带 comments (后端 getPost include comments)
export interface PostWithComments extends Post {
  comments?: Comment[];
}

export interface ListPostsParams {
  page?: number;
  limit?: number;
  sort?: SortType;
  tag?: string;
  author?: number;
}

export interface SearchPostsParams {
  keyword: string;
  page?: number;
  pageSize?: number;
  sort?: SortType;
}

export interface CreatePostBody {
  title: string;
  content?: string;
  coverImage?: string;
  images?: string[];
  genre: Genre;
  tags: string[];
  structuredData?: Object;
}

export interface CreateCommentBody {
  content: string;
  parentId?: number;
  isFact?: boolean;
}

```

```

}
export interface LoginBody {
  openId: string;
  nickname?: string;
  avatar?: string;
}
export interface LoginResult {
  token: string;
  user: User;
}
// 华为账号登录: 用 Authorization Code 换取登录态
export interface HuaweiExchangeBody {
  code: string;
}
// 腾讯云 COS 直传签名: 后端签发预签名 PUT URL, 前端直传二进制
export interface UploadSignature {
  url: string; // 预签名 PUT URL (前端直传目标)
  key: string; // 对象 Key
  cdnUrl: string; // 直传后公开可读时访问的 URL
  viewUrl: string; // GET 预签名可读链接 (私有桶也能加载, 展示与存储一律用这个)
  contentType: string; // 前端 PUT 必须带上的 Content-Type (需与签名一致)
  mode?: 'cos' | 'local'; // 上传模式: cos=直传腾讯云(外网URL); local=直传后端 /v1/upload
}
// 解析「前端 PUT 直传目标 URL」:
// - cos 模式: 直传后端下发的外部预签名 URL (如 myqcloud.com)。
// - local 模式: 上传必须走 rport(BASE_URL=127.0.0.1:3000), 与所有 API 调用一致;
// 而展示用的 viewUrl 仍用 LAN IP(backendPublicUrl)--因为模拟器 Image/WebView 不走 rport
// 严禁在 local 模式直接用 sig.url (LAN IP), 那会因模拟器 http 模块连不上而上传失败。
export function resolveUploadPutUrl(sig: UploadSignature): string {
  if (sig.mode === 'cos') {
    return sig.url;
  }
  if (sig.mode === 'local' || !sig.url.includes('myqcloud.com')) {
    const pathAndQuery: string = sig.url.replace(/^https?:\/\/[^/]+\//, '');
    return BASE_URL + pathAndQuery;
  }
  return sig.url;
}
// 拼接 query string (跳过 undefined/null/空串)
function toQuery(fields: Array<[string, string | number | undefined]>): string {
  const parts: string[] = [];
  for (const f of fields) {
    const v = f[1];
    if (v !== undefined && v !== null && v !== '') {
      parts.push(f[0] + '=' + encodeURIComponent(String(v)));
    }
  }
  return parts.length > 0 ? '?' + parts.join('&') : '';
}
// ===== 帖子 =====
// GET /v1/posts?page=&limit=&sort=hot|latest|recommend&tag=&author=
export function listPosts(params: ListPostsParams = {}): Promise<ListResult<Post>> {
  const path = '/v1/posts' + toQuery([
    ['page', params.page],
    ['limit', params.limit],
    ['sort', params.sort],
    ['tag', params.tag],
    ['author', params.author],
  ]);
  return api.get<ListResult<Post>>(path);
}

```

```

// GET /v1/posts/hotspot?windowHours=&page=&limit=&tag=
// 时窗加权热帖 (综合点赞/评论/收藏, 按时效衰减)
export interface HotPostsParams {
  windowHours?: number;
  page?: number;
  limit?: number;
  tag?: string;
}
export function listHotPosts(params: HotPostsParams = {}): Promise<ListResult<Post>> {
  const path: string = '/v1/posts/hotspot' + toQuery([
    ['windowHours', params.windowHours],
    ['page', params.page],
    ['limit', params.limit],
    ['tag', params.tag],
  ]);
  return api.get<ListResult<Post>>(path);
}
// GET /v1/posts/:id (含 comments)
export function getPost(id: number): Promise<PostWithComments> {
  return api.get<PostWithComments>('/v1/posts/' + id);
}
// GET /v1/posts/following?page=&limit=&sort=hot|latest|recommend&tag=
// 关注流: 仅当前用户关注的人的公开帖 (需登录, request 自动带 Bearer)
export function listFollowingPosts(params: ListPostsParams = {}): Promise<ListResult<Post>> {
  const path: string = '/v1/posts/following' + toQuery([
    ['page', params.page],
    ['limit', params.limit],
    ['sort', params.sort],
    ['tag', params.tag],
  ]);
  return api.get<ListResult<Post>>(path);
}
// GET /v1/posts?keyword=&page=&limit=&sort= (复用 /v1/posts 端点, keyword 选填)
export function searchPosts(params: SearchPostsParams): Promise<ListResult<Post>> {
  const path = '/v1/posts' + toQuery([
    ['keyword', params.keyword],
    ['page', params.page],
    ['limit', params.pageSize],
    ['sort', params.sort],
  ]);
  return api.get<ListResult<Post>>(path);
}
// POST /v1/posts (MVP 直接发布)
export function createPost(body: CreatePostBody): Promise<Post> {
  return api.post<Post>('/v1/posts', body);
}
// 编辑帖子 (仅本人) : PUT /v1/posts/:id
export interface UpdatePostBody {
  title?: string;
  content?: string;
  coverImage?: string;
  images?: string[];
  tags?: string[];
  structuredData?: Object;
}
export function updatePost(id: number, body: UpdatePostBody): Promise<Post> {
  return api.put<Post>('/v1/posts/' + id, body);
}
// DELETE /v1/posts/:id (仅本人)

```

```

export async function deletePost(id: number): Promise<void> {
  await api.del<null>('/v1/posts/' + id);
}
// ===== 评论 =====
// GET /v1/posts/:id/comments?page=
export function listComments(postId: number, page?: number): Promise<ListResult<
Comment>> {
  const path = '/v1/posts/' + postId + '/comments' + toQuery([[ 'page', page ]]);
  return api.get<ListResult<Comment>>(path);
}
// POST /v1/posts/:id/comments
export function createComment(postId: number, body: CreateCommentBody): Promise<
Comment> {
  return api.post<Comment>('/v1/posts/' + postId + '/comments', body);
}
// DELETE /v1/comments/:id (仅本人)
export async function deleteComment(id: number): Promise<void> {
  await api.del<null>('/v1/comments/' + id);
}
// POST /v1/comments/:id/up (评论点赞, 幂等)
export async function upComment(id: number): Promise<void> {
  await api.post<null>('/v1/comments/' + id + '/up');
}
// DELETE /v1/comments/:id/up (取消评论点赞, 幂等)
export async function cancelUpComment(id: number): Promise<void> {
  await api.del<null>('/v1/comments/' + id + '/up');
}
// ===== 互动 =====
// POST/DELETE /v1/posts/:id/up
export async function upPost(id: number): Promise<void> {
  await api.post<null>('/v1/posts/' + id + '/up');
}
export async function cancelUpPost(id: number): Promise<void> {
  await api.del<null>('/v1/posts/' + id + '/up');
}
// POST/DELETE /v1/posts/:id/bookmark
export async function bookmarkPost(id: number): Promise<void> {
  await api.post<null>('/v1/posts/' + id + '/bookmark');
}
export async function cancelBookmarkPost(id: number): Promise<void> {
  await api.del<null>('/v1/posts/' + id + '/bookmark');
}
// ===== 标签 =====
// GET /v1/tags
export function listTags(): Promise<Tag[]> {
  return api.get<Tag[]>('/v1/tags');
}
// 关注标签 (加入圈子)
// POST /v1/tags/:name/follow
export async function followTag(name: string): Promise<Tag> {
  return api.post<Tag>('/v1/tags/' + encodeURIComponent(name) + '/follow');
}
// 取消关注标签 (退出圈子)
// DELETE /v1/tags/:name/follow
export async function unfollowTag(name: string): Promise<Tag> {
  return api.del<Tag>('/v1/tags/' + encodeURIComponent(name) + '/follow');
}
// 当前用户关注的标签名列表 (圈子已加入)
// GET /v1/auth/me/followed-tags
export function listFollowedTags(): Promise<string[]> {
  return api.get<string[]>('/v1/auth/me/followed-tags');
}

```

```

// ===== 用户 / 鉴权 =====
// POST /v1/auth/login (成功后由调用方 auth.ts 统一持久化 session)
export function login(body: LoginBody): Promise<LoginResult> {
  return api.post<LoginResult>('/v1/auth/login', body);
}
// POST /v1/auth/huawei/exchange (华为 Authorization Code 换登录态; 返回结构与 login 一致)
export function exchangeHuaweiCode(body: HuaweiExchangeBody): Promise<LoginResult> {
  return api.post<LoginResult>('/v1/auth/huawei/exchange', body);
}
// GET /v1/auth/me
export function getMe(): Promise<User> {
  return api.get<User>('/v1/auth/me');
}
// PUT /v1/auth/me
export interface UpdateMeBody {
  nickname?: string;
  avatar?: string;
  bio?: string;
  gender?: number;
}
export function updateMe(body: UpdateMeBody): Promise<User> {
  return api.put<User>('/v1/auth/me', body);
}
// 账号注销 (软删): POST /v1/auth/me/deactivate, 无参, 自动带 Bearer。
// 成功后前端自行清 token (见 SettingsPage 的 doDeactivate)。
export function deactivateAccount(): Promise<void> {
  return api.post<void>('/v1/auth/me/deactivate');
}
// GET /v1/auth/me/bookmarks?page=&limit= (我的收藏帖子列表, 分页)
export function listMyBookmarks(page: number = 1, limit: number = 20): Promise<ListResult<Post>> {
  const path = '/v1/auth/me/bookmarks' + toQuery([[ 'page', page ], [ 'limit', limit ]]);
  return api.get<ListResult<Post>>(path);
}
// GET /v1/auth/me/likes?page=&limit= (我赞过的帖子列表, 分页)
export function listMyLikedPosts(page: number = 1, limit: number = 20): Promise<ListResult<Post>> {
  const path = '/v1/auth/me/likes' + toQuery([[ 'page', page ], [ 'limit', limit ]]);
  return api.get<ListResult<Post>>(path);
}
// GET /v1/auth/me/commented?page=&limit= (我评论过的帖子列表, 分页)
export function listMyCommentedPosts(page: number = 1, limit: number = 20): Promise<ListResult<Post>> {
  const path = '/v1/auth/me/commented' + toQuery([[ 'page', page ], [ 'limit', limit ]]);
  return api.get<ListResult<Post>>(path);
}
// ===== 账号绑定管理 =====
// GET /v1/account/bindings (当前用户全部绑定, harmony 主账号置顶)
export function listBindings(): Promise<BindingItem[]> {
  return api.get<BindingItem[]>('/v1/account/bindings');
}
// POST /v1/account/bindings (绑定第三方, code 变体)
export function bindAccount(provider: Provider, code: string): Promise<BindingItem> {
  const body: BindAccountBody = { provider, code };
  return api.post<BindingItem>('/v1/account/bindings', body);
}

```

```

// DELETE /v1/account/bindings/:provider (解绑第三方, harmony 不可解绑)
export function unbindAccount(provider: Provider): Promise<UnbindResult> {
  return api.del<UnbindResult>('/v1/account/bindings/' + provider);
}
// ===== 消息通知 =====
// GET /v1/notifications?page=&limit= (当前用户通知列表, 分页)
export function listNotifications(page: number = 1, limit: number = 20): Promise<ListResult<NotificationItem>> {
  const path = '/v1/notifications' + toQuery([[ 'page', page ], [ 'limit', limit ]]);
  return api.get<ListResult<NotificationItem>>(path);
}
// GET /v1/notifications/unread-count (未读总数, 用于 Tab 红点)
export function getUnreadCount(): Promise<UnreadCount> {
  return api.get<UnreadCount>('/v1/notifications/unread-count');
}
// POST /v1/notifications/:id/read (标记单条已读)
export async function markNotificationRead(id: number): Promise<void> {
  await api.post<null>('/v1/notifications/' + id + '/read');
}
// POST /v1/notifications/read-all (全部已读)
export async function markAllRead(): Promise<void> {
  await api.post<null>('/v1/notifications/read-all');
}
// ===== 上传 =====
// POST /v1/upload/token → 返回腾讯云 COS 预签名 PUT URL (前端直传)
export interface UploadSignatureBody {
  contentType?: string;
}
export function getUploadSignature(contentType: string = 'image/jpeg'): Promise<UploadSignature> {
  const body: UploadSignatureBody = { contentType };
  return api.post<UploadSignature>('/v1/upload/token', body);
}
// ===== 搜索历史 / 热搜词 =====
// 搜索历史项 (后端返回)
export interface SearchHistoryItem {
  id: number;
  keyword: string;
  createdAt: string;
}
// 热搜词项 (后端返回)
export interface HotKeywordItem {
  keyword: string;
  count: number;
}
// 搜索历史列表响应
export interface SearchHistoryResult {
  list: SearchHistoryItem[];
}
// 热搜词列表响应
export interface HotKeywordResult {
  list: HotKeywordItem[];
}
// 记录搜索历史的请求体
export interface RecordSearchHistoryBody {
  keyword: string;
}
// POST /v1/search/history 记录搜索历史 (前端搜索成功后调用)
export async function recordSearchHistory(keyword: string): Promise<void> {
  const body: RecordSearchHistoryBody = { keyword };
  await api.post<null>('/v1/search/history', body);
}

```



```

}
// GET /v1/search/history 搜索历史列表 (返回关键词数组, 按时间倒序)
export async function listSearchHistory(): Promise<string[]> {
  const res: SearchHistoryResult = await api.get<SearchHistoryResult>('/v1/search/history');
  const items: SearchHistoryItem[] = res.list;
  const keywords: string[] = [];
  for (let i = 0; i < items.length; i++) {
    keywords.push(items[i].keyword);
  }
  return keywords;
}
// DELETE /v1/search/history 清除搜索历史
export async function clearSearchHistory(): Promise<void> {
  await api.del<null>('/v1/search/history');
}
// GET /v1/search/hot?limit=10 热搜词列表 (返回关键词数组, 按搜索次数降序)
export async function listHotKeywords(limit?: number): Promise<string[]> {
  const path: string = '/v1/search/hot' + toQuery([[ 'limit', limit ]]);
  const res: HotKeywordResult = await api.get<HotKeywordResult>(path);
  const items: HotKeywordItem[] = res.list;
  const keywords: string[] = [];
  for (let i = 0; i < items.length; i++) {
    keywords.push(items[i].keyword);
  }
  return keywords;
}
// ===== 关注 / 他人主页 =====
// GET /v1/users/:id (他人/自己资料, 含聚合计数与关系态)
// 注意: 不要传 'me'--'me' 会被 /v1/users 下的 authRouter 抢先命中返回原始 user (缺计数),
export function getUserProfile(id: number): Promise<UserProfile> {
  return api.get<UserProfile>('/v1/users/' + id);
}
// POST /v1/users/:id/follow (关注, 幂等; 后端校验不可自关)
export async function followUser(id: number): Promise<void> {
  await api.post<null>('/v1/users/' + id + '/follow');
}
// DELETE /v1/users/:id/follow (取消关注, 幂等)
export async function unfollowUser(id: number): Promise<void> {
  await api.del<null>('/v1/users/' + id + '/follow');
}
// GET /v1/users/:id/following?page=&limit= (关注列表, 分页)
export function listFollowing(id: number, page: number = 1, limit: number = 20):
  Promise<FollowListResult> {
  const path = '/v1/users/' + id + '/following' + toQuery([[ 'page', page ], [ 'limit', limit ]]);
  return api.get<FollowListResult>(path);
}
// GET /v1/users/:id/followers?page=&limit= (粉丝列表, 分页)
export function listFollowers(id: number, page: number = 1, limit: number = 20):
  Promise<FollowListResult> {
  const path = '/v1/users/' + id + '/followers' + toQuery([[ 'page', page ], [ 'limit', limit ]]);
  return api.get<FollowListResult>(path);
}
// ===== 举报 =====
// POST /v1/posts/:id/report (举报帖子)
export async function reportPost(id: number, body: ReportBody): Promise<void> {
  await api.post<null>('/v1/posts/' + id + '/report', body);
}
// POST /v1/comments/:id/report (举报评论)
export async function reportComment(id: number, body: ReportBody): Promise<void>

```

```

    {
      await api.post<null>('/v1/comments/' + id + '/report', body);
    }
  }
  // ===== 内容审核 (admin) =====
  // GET /v1/admin/posts/pending?page=&limit= 待审核帖子列表 (含举报记录)
  export function getPendingPosts(page: number = 1, limit: number = 20): Promise<ListResult<PendingPost>> {
    const path: string = '/v1/admin/posts/pending' + toQuery([[ 'page', page ], [ 'limit', limit ]]);
    return api.get<ListResult<PendingPost>>(path);
  }
  // POST /v1/admin/posts/:id/moderate 请求体
  export interface ModerateBody {
    action: ModerateAction;
    reason?: string;
  }
  // POST /v1/admin/posts/:id/moderate {action: approve|reject, reason?}
  export async function moderatePost(id: number, action: ModerateAction, reason?: string): Promise<void> {
    const body: ModerateBody = { action, reason };
    await api.post<null>('/v1/admin/posts/' + id + '/moderate', body);
  }
  // ===== 通知偏好 =====
  // GET /v1/me/notification-prefs
  export function getNotificationPrefs(): Promise<NotificationPrefs> {
    return api.get<NotificationPrefs>('/v1/me/notification-prefs');
  }
  // PUT /v1/me/notification-prefs
  export function updateNotificationPrefs(body: NotificationPrefs): Promise<NotificationPrefs> {
    return api.put<NotificationPrefs>('/v1/me/notification-prefs', body);
  }
  // ===== 拉黑管理 =====
  // GET /v1/me/blocklist?page=&limit=
  export function listBlocked(page: number = 1, limit: number = 20): Promise<ListResult<BlockUser>> {
    const path: string = '/v1/me/blocklist' + toQuery([[ 'page', page ], [ 'limit', limit ]]);
    return api.get<ListResult<BlockUser>>(path);
  }
  // POST /v1/me/block/:blockedId
  export async function blockUser(blockedId: number): Promise<void> {
    await api.post<null>('/v1/me/block/' + blockedId);
  }
  // DELETE /v1/me/block/:blockedId
  export async function unblockUser(blockedId: number): Promise<void> {
    await api.del<null>('/v1/me/block/' + blockedId);
  }
  // ===== 隐私设置 =====
  // GET /v1/me/privacy
  export function getPrivacySettings(): Promise<PrivacySettingsData> {
    return api.get<PrivacySettingsData>('/v1/me/privacy');
  }
  // PUT /v1/me/privacy
  export function updatePrivacySettings(body: PrivacySettingsData): Promise<PrivacySettingsData> {
    return api.put<PrivacySettingsData>('/v1/me/privacy', body);
  }
  // ===== 私信 =====
  // ArkTS V1 严格模式: 禁止内联对象字面量类型, 须用具名 interface
  export interface ConversationsResult {
    list: Conversation[];
  }

```

```

}
export interface DmUnreadResult {
  count: number;
}
export interface MessagesResult {
  list: DmMessage[];
}
// GET /v1/messages/conversations -- 会话列表
export function getConversations(): Promise<ConversationsResult> {
  return api.get<ConversationsResult>('/v1/messages/conversations');
}
// GET /v1/messages/unread -- 私信未读总数
export function getDmUnreadCount(): Promise<DmUnreadResult> {
  return api.get<DmUnreadResult>('/v1/messages/unread');
}
// GET /v1/messages/:userId?page= -- 与某用户私信历史 (旧→新)
export function getMessages(userId: number, page: number = 1): Promise<MessagesResult> {
  return api.get<MessagesResult>('/v1/messages/' + userId + '?page=' + page);
}
// POST /v1/messages/:userId/read -- 标记与该用户私信已读
export async function markMessagesRead(userId: number): Promise<void> {
  await api.post<null>('/v1/messages/' + userId + '/read');
}
// POST /v1/messages { receiverId, content } -- 发送私信
interface SendMessageBody {
  receiverId: number;
  content: string;
}
export async function sendMessage(receiverId: number, content: string): Promise<DmMessage> {
  return api.post<DmMessage>('/v1/messages', { receiverId, content } as SendMessageBody);
}
// ===== 辩论投票 =====
// POST /v1/posts/:id/vote { choice: 'A' | 'B' }
interface VoteDebateBody {
  choice: string;
}
export function voteDebate(postId: number, choice: string): Promise<DebateVoteResult> {
  const body: VoteDebateBody = { choice: choice };
  return api.post<DebateVoteResult>('/v1/posts/' + postId + '/vote', body);
}
// ===== 数据导出 =====
// GET /v1/me/export
export function exportUserData(): Promise<UserExportData> {
  return api.get<UserExportData>('/v1/me/export');
}

// ===== ./entry/src/main/ets/utils/actionIcons.ets =====
// 动作 / 状态 / 通知类型 → 线条图标资源 映射 (Apple 风格极简线条 Logo)
// 与 tagIcons.ets 同源: 用 Map<string, Resource> + 字面量 $r(),
// 既规避 ArkTS 索引访问禁令, 又能在编译期校验资源确实存在。
import { NotificationType } from '../models/types';
const ACTION_ICONS: Map<string, Resource> = new Map<string, Resource>([
  // 动作
  ['like', $r('app.media.action_like')],
  ['comment', $r('app.media.action_comment')],
  ['bookmark', $r('app.media.action_bookmark')],
  ['share', $r('app.media.action_share')],
  ['follow', $r('app.media.action_follow')],

```

```

// 通知类型
['system', $r('app.media.notification_system')],
// 状态 / 空态
['lock', $r('app.media.status_lock')],
['warning', $r('app.media.status_warning')],
['guide', $r('app.media.empty_guide')],
['inbox', $r('app.media.empty_inbox')],
['search', $r('app.media.empty_search')],
// 导航 / 控件
['back', $r('app.media.nav_back')],
['close', $r('app.media.close_x')],
['chevronDown', $r('app.media.chevron_down')],
['chevronUp', $r('app.media.chevron_up')],
['phone', $r('app.media.device_phone')],
['book', $r('app.media.book_doc')],
['check', $r('app.media.check_round')],
]);
// 按动作名解析线条图标资源; 未命中回退 close (x)
export function resolveActionIcon(name: string): Resource {
  const found: Resource | undefined = ACTION_ICONS.get(name);
  if (found !== undefined) {
    return found;
  }
  return $r('app.media.close_x');
}
// 通知类型 → 图标资源 (ArkTS V1 不支持 obj[key] 索引访问, 用 switch 分发)
export function notifIconRes(type: NotificationType | undefined): Resource {
  const t: NotificationType = type ?? 'system';
  switch (t) {
    case 'comment':
      return $r('app.media.action_comment');
    case 'up':
      return $r('app.media.action_like');
    case 'bookmark':
      return $r('app.media.action_bookmark');
    case 'follow':
      return $r('app.media.action_follow');
    default:
      return $r('app.media.notification_system');
  }
}

// ===== ./entry/src/main/ets/utils/auth.ets =====
// 统一登录态 (横切地基): 持久化 token + 当前用户, 供全应用复用。
// 真登录已接入: 无登录态时 ensureLogin 不再静默建号, 而是跳到 LoginPage 由用户显式选择
// 「华为一键登录」或「其他方式登录 (开发桩)」。
import { login, LoginResult } from '../services/api';
import { User } from '../models/types';
import { router } from '@kit.ArkUI';
// 进程级持久化 (基于 Preferences, 重启/重进页面不丢)。必须在任何 AppStorage 读写前注册一次。
PersistentStorage.persistProp('authToken', '');
PersistentStorage.persistProp('authUserId', '');
PersistentStorage.persistProp('authNickname', '');
PersistentStorage.persistProp('authAvatar', '');
PersistentStorage.persistProp('authBio', ''); // 个人简介 (编辑资料回写, 避免再进)
PersistentStorage.persistProp('authGender', 0); // 性别: 0=保密/未设 1=男 2=女
PersistentStorage.persistProp('authIsAdmin', false); // 管理员标识 (后端 env.adminUser
s 计算, 登录时落地)
PersistentStorage.persistProp('unreadMessageCount', 0); // 消息 Tab 未读红点
PersistentStorage.persistProp('unreadDmCount', 0); // 私信未读红点 (与通知红点合并显示)
PersistentStorage.persistProp('privacyAgreed', '0'); // 首启隐私协议同意标记 ('1'=已同意)
export function getToken(): string {

```

```

    return (AppStorage.get<string>('authToken') ?? '') as string;
}
export function getIsAdmin(): boolean {
    return (AppStorage.get<boolean>('authIsAdmin') ?? false) as boolean;
}
export function getUserId(): number {
    const v: string = (AppStorage.get<string>('authUserId') ?? '') as string;
    const n: number = parseInt(v, 10);
    return isNaN(n) ? 0 : n;
}
export function getCurrentUser(): User | null {
    const id: number = getUserId();
    if (id === 0) {
        return null;
    }
    const nickname: string = (AppStorage.get<string>('authNickname') ?? '') as string;
    const avatar: string = (AppStorage.get<string>('authAvatar') ?? '') as string;
    const bio: string = (AppStorage.get<string>('authBio') ?? '') as string;
    const gender: number = (AppStorage.get<number>('authGender') ?? 0) as number;
    const isAdmin: boolean = (AppStorage.get<boolean>('authIsAdmin') ?? false) as boolean;
    return { id, nickname, avatar, bio, gender, isAdmin } as User;
}
export function setSession(token: string, user: User): void {
    AppStorage.setOrCreate('authToken', token);
    AppStorage.setOrCreate('authUserId', String(user.id ?? ''));
    AppStorage.setOrCreate('authNickname', user.nickname ?? '');
    AppStorage.setOrCreate('authAvatar', user.avatar ?? '');
    AppStorage.setOrCreate('authBio', user.bio ?? '');
    AppStorage.setOrCreate('authGender', user.gender ?? 0);
    AppStorage.setOrCreate('authIsAdmin', user.isAdmin ? true : false);
}
// 清登录态: token / 用户信息 / 管理员态全部清空 (登出、账号注销共用) 。
export function clearSession(): void {
    AppStorage.setOrCreate('authToken', '');
    AppStorage.setOrCreate('authUserId', '');
    AppStorage.setOrCreate('authNickname', '');
    AppStorage.setOrCreate('authAvatar', '');
    AppStorage.setOrCreate('authBio', '');
    AppStorage.setOrCreate('authGender', 0);
    AppStorage.setOrCreate('authIsAdmin', false);
}
// 确保登录态: 有登录态直接返回当前用户; 无登录态则跳到 LoginPage 由用户显式登录, 并返回 null。
// 不再静默用 dev 桩建号 (原 MVP 行为), 避免真实登录入口被绕过。
export async function ensureLogin(): Promise<User | null> {
    const existing: User | null = getCurrentUser();
    if (existing && getToken()) {
        return existing;
    }
    // 防递归: LoginPage 自身 (如「其他方式登录」) 调用时不再重复 pushUrl
    try {
        const state = router.getState();
        if (state && state.path !== 'pages/LoginPage') {
            router.pushUrl({ url: 'pages/LoginPage' });
        }
    } catch (e) {
        // router 未就绪等异常时直接尝试跳转, 不阻断
        try {
            router.pushUrl({ url: 'pages/LoginPage' });
        } catch (_) { /* ignore */ }
    }
}

```

```

    return null;
}
// 开发/预览兜底：仅由 LoginPage 的「其他方式登录」手动调用，静默桩逻辑集中在此处，
// 不再由 ensureLogin 自动触发。真机走华为账号一键登录 (exchangeHuaweiCode) 。
export async function loginWithDevStub(): Promise<User> {
    const res: LoginResult = await login({ openId: 'dev-seed-openid', nickname: '科技老张' });
    setSession(res.token, res.user);
    return res.user;
}
// 幂等兜底：确保在 AppStorage 读写前已注册持久化 (Index aboutToAppear 调用一次) 。
export function initAuthStorage(): void {
    PersistentStorage.persistProp('authToken', '');
    PersistentStorage.persistProp('authUserId', '');
    PersistentStorage.persistProp('authNickname', '');
    PersistentStorage.persistProp('authAvatar', '');
    PersistentStorage.persistProp('authBio', '');
    PersistentStorage.persistProp('authGender', 0);
    PersistentStorage.persistProp('authIsAdmin', false);
    PersistentStorage.persistProp('unreadMessageCount', 0);
    PersistentStorage.persistProp('unreadDmCount', 0);
    PersistentStorage.persistProp('privacyAgreed', '0');
}
// 首启隐私协议：是否已同意 ('1'=已同意)。未同意则 Index 弹窗拦截。
export function isPrivacyAgreed(): boolean {
    return (AppStorage.get<string>('privacyAgreed') ?? '0') === '1';
}
// 首启隐私协议：标记为已同意并持久化 (PersistentStorage 自动落盘) 。
export function setPrivacyAgreed(): void {
    AppStorage.setOrCreate('privacyAgreed', '1');
}
// 华为登录结果落地：语义化封装 setSession，供 LoginPage 在拿到 LoginResult 后调用。
// 与 openId 登录共用同一套登录态 (F2: 直接调 setSession 复用) 。
export function loginByHuaweiResult(result: LoginResult): void {
    setSession(result.token, result.user);
}

// ===== ./entry/src/main/ets/utils/breakpoint.ets =====
// 响应式断点系统：根据窗口宽度返回适配参数
// 用于 Grid 列数、字体缩放、间距调整等
// 断点定义
const BREAKPOINT_SM = 600;    // vp: 手机竖屏
const BREAKPOINT_MD = 840;    // vp: 手机横屏 / 平板竖屏
const BREAKPOINT_LG = 1024;   // vp: 平板横屏 / 桌面
export enum Breakpoint {
    SM = 'sm',
    MD = 'md',
    LG = 'lg',
}
// 根据窗口宽度获取当前断点
export function getBreakpoint(width: number): Breakpoint {
    if (width < BREAKPOINT_SM) {
        return Breakpoint.SM;
    }
    if (width < BREAKPOINT_MD) {
        return Breakpoint.MD;
    }
    return Breakpoint.LG;
}
// 根据断点获取 Grid 列数
export function getGridColumn(bp: Breakpoint): number {
    switch (bp) {

```

```

        case Breakpoint.SM:
            return 2;
        case Breakpoint.MD:
            return 3;
        case Breakpoint.LG:
            return 4;
        default:
            return 2;
    }
}
// 根据断点获取 Grid 列模板字符串
export function getGridTemplate(bp: Breakpoint): string {
    const cols: number = getGridColumns(bp);
    return '1fr '.repeat(cols).trim();
}

// ===== ./entry/src/main/ets/utils/cardTokens.ets =====
// 卡片设计令牌：统一全应用所有「卡片」场景的圆角 / 内边距 / 阴影 / 网格间距
// 用途：消除 PostCard、圈子卡片等多处重复且不一致的样式数值，确保视觉一致
// 接入页面：PostCard、CircleTab、HomeTab、ProfilePage、SearchResultPage
// 卡片圆角 (Apple大圆角，较 16 略收紧一档)
export const CARD_RADIUS: number = 14;
// 卡片内边距 (比旧值 12 再收紧，让封面图更贴近卡片左右边框，更紧凑)
export const CARD_PADDING: number = 6;
// 卡片内封面图圆角 (内嵌于卡片，比卡片圆角略小)
export const CARD_COVER_RADIUS: number = 8;
// 卡片统一投影：浅色模式淡灰、深色模式同样适用 (颜色含透明度，深浅模式自动过渡)
export const CARD_SHADOW: ShadowOptions = {
    radius: 12,
    color: '#0D000000',
    offsetY: 2
};
// 网格列间距 / 行间距 (再收紧一档，让卡片排布更紧凑)
export const GRID_GAP: number = 6;
// 网格外边距 (再收紧一档)
export const GRID_PADDING: number = 8;
// 卡片内部元素纵向间距 (标题↔作者↔互动行的间距，比旧值 10 收紧)
export const CARD_INNER_GAP: number = 8;

// ===== ./entry/src/main/ets/utils/data-source.ets =====
// 通用 LazyForEach 数据源，支持列表数据的懒加载渲染
// 替代 ForEach，在大量数据场景下显著提升性能
export class BasicDataSource<T> implements IDataSource {
    private listeners: DataChangeListener[] = [];
    private dataArray: T[] = [];
    totalCount(): number {
        return this.dataArray.length;
    }
    getData(index: number): T {
        return this.dataArray[index] as T;
    }
    registerDataChangeListener(listener: DataChangeListener): void {
        if (this.listeners.indexOf(listener) < 0) {
            this.listeners.push(listener);
        }
    }
    unregisterDataChangeListener(listener: DataChangeListener): void {
        const pos: number = this.listeners.indexOf(listener);
        if (pos >= 0) {
            this.listeners.splice(pos, 1);
        }
    }
}

```

```

// 通知数据变更（用于刷新场景）
notifyDataReload(): void {
    this.listeners.forEach((listener: DataChangeListener) => {
        listener.onDataReloaded();
    });
}
notifyDataAdd(index: number): void {
    this.listeners.forEach((listener: DataChangeListener) => {
        listener.onDataAdd(index);
    });
}
notifyDataChange(index: number): void {
    this.listeners.forEach((listener: DataChangeListener) => {
        listener.onDataChange(index);
    });
}
notifyDataDelete(index: number): void {
    this.listeners.forEach((listener: DataChangeListener) => {
        listener.onDataDelete(index);
    });
}
// 清空数据并通知
clearData(): void {
    this.dataArray = [];
    this.notifyDataReload();
}
// 获取数据长度（用于空状态判断）
getLength(): number {
    return this.dataArray.length;
}
// 设置数据
setData(data: T[]): void {
    this.dataArray = data;
    this.notifyDataReload();
}
// 追加数据
appendData(data: T[]): void {
    for (let i = 0; i < data.length; i++) {
        this.dataArray.push(data[i]);
        this.notifyDataAdd(this.dataArray.length - 1);
    }
}
// 替换全部数据
replaceData(data: T[]): void {
    this.dataArray = data;
}
// 获取原始数据（用于外部访问）
getRawData(): T[] {
    return this.dataArray;
}
}

// ===== ./entry/src/main/ets/utils/format.ets =====
export function formatCount(n?: number): string {
    if (!n) {
        return '0';
    }
    if (n >= 10000) {
        return (n / 10000).toFixed(1) + 'w';
    }
    return n.toString();
}

```



```

export function timeAgo(iso?: string): string {
  if (!iso) {
    return '';
  }
  const diff = Date.now() - new Date(iso).getTime();
  const m = Math.floor(diff / 60000);
  if (m < 1) {
    return '刚刚';
  }
  if (m < 60) {
    return m + '分钟前';
  }
  const h = Math.floor(m / 60);
  if (h < 24) {
    return h + '小时前';
  }
  const d = Math.floor(h / 24);
  if (d < 30) {
    return d + '天前';
  }
  return new Date(iso).toLocaleDateString();
}

// ===== ./entry/src/main/ets/utils/image.ets =====
// 图片压缩: 选图后长边压缩 + 质量 80% 转 jpeg (详见技术文档 5.3)。
// 同时返回 PixelMap (本地预览) 与 bytes (上传 COS 用), 一举两得。
import { image } from '@kit.ImageKit';
import { fileIo } from '@kit.CoreFileKit';
export interface CompressedImage {
  preview: image.PixelMap; // 本地预览用
  bytes: ArrayBuffer; // 上传二进制用 (jpeg)
}
export async function compressImage(uri: string): Promise<CompressedImage> {
  // 相册返回的 photo:// URI 是沙箱受限 URI, 必须经由文件描述符打开,
  // 直接用 image.createImageSource(uri) 在某些 SDK 会返回 undefined (导致 createPixelMap
) 。
  const file = fileIo.openSync(uri, fileIo.OpenMode.READ_ONLY);
  let source: image.ImageSource;
  try {
    source = image.createImageSource(file.fd);
  } catch (e) {
    fileIo.closeSync(file);
    throw new Error((e as Error).message);
  }
  try {
    const pixelMap: image.PixelMap = await source.createPixelMap();
    const packer: image.ImagePacker = image.createImagePacker();
    try {
      const bytes: ArrayBuffer = await packer.packToData(pixelMap, {
        format: 'image/jpeg',
        quality: 80,
      });
    }
    return { preview: pixelMap, bytes };
  } finally {
    await packer.release();
  }
  finally {
    source.release();
    fileIo.closeSync(file);
  }
}

// 头像专用压缩: 限制最长边 maxEdge (默认 512px), 避免本地 base64 兜底时体积过大, 质量 80%。

```

```
// 其余逻辑与 compressImage 一致 (file descriptor 打开 photo:// URI) 。
export async function compressAvatar(uri: string, maxEdge: number = 512): Promise<CompressedImage> {
  const file = fileIo.openSync(uri, fileIo.OpenMode.READ_ONLY);
  let source: image.ImageSource;
  try {
    source = image.createImageSource(file.fd);
  } catch (e) {
    fileIo.closeSync(file);
    throw new Error((e as Error).message);
  }
  try {
    const pixelMap: image.PixelMap = await source.createPixelMap();
    const info = pixelMap.getImageInfoSync();
    const longEdge: number = Math.max(info.size.width, info.size.height);
    if (longEdge > maxEdge) {
      const ratio: number = maxEdge / longEdge;
      await pixelMap.scale(ratio, ratio);
    }
    const packer: image.ImagePacker = image.createImagePacker();
    try {
      const bytes: ArrayBuffer = await packer.packToData(pixelMap, {
        format: 'image/jpeg',
        quality: 80,
      });
      return { preview: pixelMap, bytes };
    } finally {
      await packer.release();
    }
  } finally {
    source.release();
    fileIo.closeSync(file);
  }
}
```

```
// ===== ./entry/src/main/ets/utils/likeStore.ets =====
```

```
// 全局点赞状态仓库 (单例)
```

```
//
```

```
// 背景: 点赞态此前只存在每个 PostCard / DetailPage 各自的 @State 里, 首页卡片点赞后,
// 详情页、个人主页「我赞过」里的同一帖子副本完全不知道, 导致跨页点赞数/已赞态不一致,
// 且「我赞过」列表在取消点赞后不会即时消失。
```

```
//
```

```
// 方案: 用一个进程级单例保存 postId -> { liked, count }, 所有展示帖子的组件在 aboutToAppear
// 时向仓库「注册并订阅」, 点赞/取消时写入仓库并广播; 订阅回调把最新值写回组件本地 @State,
// 触发重渲染。这是 ArkUI V1 严格模式下最稳的跨组件实时同步方式 (不依赖 @Observed 深层观察) 。
```

```
export class LikeInfo {
  liked: boolean = false;
  count: number = 0;
  constructor(liked: boolean, count: number) {
    this.liked = liked;
    this.count = count;
  }
}
```

```
type LikeListener = (postId: number) => void;
class LikeStoreImpl {
```

```
  private map: Map<number, LikeInfo> = new Map<number, LikeInfo>();
```

```
  private listeners: LikeListener[] = [];
```

```
  // 仅当仓库中尚无该帖子时灌入初始态 (不覆盖本地已发生的点赞操作)
```

```
  ensure(postId: number, liked: boolean, count: number): void {
```

```
    if (!this.map.has(postId)) {
      this.map.set(postId, new LikeInfo(liked, count));
    }
  }
```

```

    }
    get(postId: number): LikeInfo | undefined {
        return this.map.get(postId);
    }
    // 乐观更新 (点赞/取消)。返回操作前的快照 (LikeInfo 实例)，便于请求失败时回滚。
    applyOptimistic(postId: number, willLike: boolean): LikeInfo {
        let info: LikeInfo | undefined = this.map.get(postId);
        if (!info) {
            info = new LikeInfo(false, 0);
            this.map.set(postId, info);
        }
        const prev: LikeInfo = new LikeInfo(info.liked, info.count);
        const delta: number = willLike ? 1 : (info.liked ? -1 : 0);
        info.liked = willLike;
        info.count = Math.max(0, info.count + delta);
        this.emit(postId);
        return prev;
    }
    // 请求失败回滚到操作前快照
    revert(postId: number, prev: LikeInfo): void {
        const info: LikeInfo | undefined = this.map.get(postId);
        if (info) {
            info.liked = prev.liked;
            info.count = prev.count;
            this.emit(postId);
        }
    }
    // 用服务器返回的绝对值覆盖 (进入详情页 / 刷新列表后校准，避免与本地乐观值漂移)
    syncFromServer(postId: number, liked: boolean, count: number): void {
        let info: LikeInfo | undefined = this.map.get(postId);
        if (!info) {
            info = new LikeInfo(liked, count);
            this.map.set(postId, info);
        } else {
            info.liked = liked;
            info.count = count;
        }
        this.emit(postId);
    }
    // 订阅某帖子点赞态变化；返回取消订阅函数 (组件 aboutToDisappear 时调用，防泄漏)
    subscribe(cb: LikeListener): () => void {
        this.listeners.push(cb);
        return (): void => {
            const idx: number = this.listeners.indexOf(cb);
            if (idx >= 0) {
                this.listeners.splice(idx, 1);
            }
        };
    }
    private emit(postId: number): void {
        for (const cb of this.listeners) {
            cb(postId);
        }
    }
}

export const likeStore: LikeStoreImpl = new LikeStoreImpl();

// ===== ./entry/src/main/ets/utils/nav.ts =====
// 详情页传参: router.getParams() 在 API 24 已弃用且返回空，改用模块级单例。
// 同 EntryAbility 内页面栈共享该模块，跳转前存 id，详情页取 id。
let _detailPostId: string = '';
export function setDetailPostId(id: string): void {

```

```

    _detailPostId = id;
  }
  export function takeDetailPostId(): string {
    const id = _detailPostId;
    _detailPostId = '';
    return id;
  }
  // 跨 Tab 跳转信号：子组件调用此函数触发 Index 切换 Tab。
  // 用递增序列号保证 @Watch 每次都能触发 (@Watch 对同值不触发)。
  // 注意：target 必须先写、signal 后写，确保 Index 的 @Watch 回调读到最新 target。
  export function emitTabSwitch(target: number): void {
    AppStorage.setOrCreate('tabSwitchTarget', target);
    const seq: number = (AppStorage.get<number>('tabSwitchSignal') ?? 0) + 1;
    AppStorage.setOrCreate('tabSwitchSignal', seq);
  }

  // ===== ./entry/src/main/ets/utils/responsive.ets =====
  // 响应式网格工具：根据屏幕宽度计算卡片列数，并监听窗口尺寸变化（折叠屏/平板/旋转）
  // 用途：让首页/个人主页/搜索/圈子的卡片网格在不同屏幕尺寸下列数自适应，避免溢出或布局错乱
  import { window } from '@kit.ArkUI';
  import { common } from '@kit.AbilityKit';
  // 根据屏幕物理宽度 (px) 计算网格列数
  // 手机竖屏 (<600vp)          → 2 列
  // 折叠屏展开/平板竖屏      → 3 列
  // 平板横屏/桌面 (>840vp)    → 4 列
  function computeGridColumnColumns(widthPx: number): number {
    const wvp: number = px2vp(widthPx);
    if (wvp >= 840) {
      return 4;
    }
    if (wvp >= 600) {
      return 3;
    }
    return 2;
  }
  // 生成 Grid 的 columnsTemplate 字符串，例如 3 列 → '1fr 1fr 1fr'
  export function gridColumnTemplate(cols: number): string {
    let tpl: string = '';
    for (let i: number = 0; i < cols; i++) {
      if (i > 0) {
        tpl += ' ';
      }
      tpl += '1fr';
    }
    return tpl;
  }
  // 注册窗口尺寸监听：尺寸变化时通过 onChange 回传最新列数；返回注销函数，须在 aboutToDisappear
  // ArkTS V1 约束：回调用具名函数类型 (size: window.Size) => void，禁止内联对象字面量
  export function observeGridColumnColumns(ctx: common.Context, onChange: (cols: number)
    => void): () => void {
    let handler: ((size: window.Size) => void) | undefined = undefined;
    window.getLastWindow(ctx)
      .then((win: window.Window) => {
        const rect = win.getWindowProperties().windowRect;
        onChange(computeGridColumnColumns(rect.width));
        handler = (size: window.Size): void => {
          onChange(computeGridColumnColumns(size.width));
        };
        win.on('windowSizeChange', handler);
      })
      .catch(() => {
        onChange(2);
      });
  }

```

```

});
return (): void => {
  if (handler !== undefined) {
    window.getLastWindow(ctx)
      .then((win: window.Window) => {
        win.off('windowSizeChange', handler);
      })
      .catch(() => { /* 忽略注销失败 */ });
  }
};
}

// ===== ./entry/src/main/ets/utils/settingsIcons.ets =====
// 设置页 → 线条图标资源映射 (Apple 风格极简线条 Logo)
// 与 tagIcons.ets / actionIcons.ets 同源: Map<string, Resource> + 字面量 $r(),
// 规避 ArkTS 索引访问禁令, 编译期校验资源存在。
const SETTINGS_ICONS: Map<string, Resource> = new Map<string, Resource>([
  // 导航 / 复用主界面线条图标
  ['back', $r('app.media.nav_back')],
  ['phone', $r('app.media.device_phone')],
  ['lock', $r('app.media.status_lock')],
  ['book', $r('app.media.book_doc')],
  ['chevronRight', $r('app.media.chevron_right')],
  // 设置项专属
  ['edit', $r('app.media.set_edit')],
  ['block', $r('app.media.set_block')],
  ['export', $r('app.media.set_export')],
  ['bell', $r('app.media.set_bell')],
  ['system', $r('app.media.set_system')],
  ['dark', $r('app.media.set_dark')],
  ['trash', $r('app.media.set_trash')],
  ['policy', $r('app.media.set_policy')],
  ['logout', $r('app.media.set_logout')],
  ['shield', $r('app.media.set_shield')],
]);
// 按名称解析设置页图标资源; 未命中回退 chevronRight (箭头), 避免空白
export function resolveSettingsIcon(name: string): Resource {
  const found: Resource | undefined = SETTINGS_ICONS.get(name);
  if (found !== undefined) {
    return found;
  }
  return $r('app.media.chevron_right');
}

// ===== ./entry/src/main/ets/utils/share.ets =====
// 分享工具 (占位实现)
// 说明: 当前线上域名尚未确定 (上线阻塞项之一), SHARE_BASE 为占位值。
// 待真实生产域名确定后, 仅需修改 SHARE_BASE 一处即可全局生效, 无需改调用点。
import pasteboard from '@ohos.pasteboard';
// TODO(上线): 替换为真实生产域名 (域名确定后改这一行即可)
const SHARE_BASE: string = 'https://bigbluebook.app';
// 帖子分享链接: 占位域名 + /post/{id}
export function buildPostShareUrl(postId: number): string {
  return SHARE_BASE + '/post/' + String(postId);
}
// 分享文案 (标题 + 链接), 用于系统分享面板接入后使用
export function buildShareText(title: string, postId: number): string {
  const t: string = title && title.length > 0 ? title : '大蓝书帖子';
  return t + ' ' + buildPostShareUrl(postId);
}
// 复制文本到系统剪贴板; 失败抛出, 由调用方 toast 兜底
export function copyToClipboard(text: string): void {

```

```

const pb = pasteboard.getSystemPasteboard();
pb.setData(pasteboard.createPlainTextData(text));
}

// ===== ./entry/src/main/ets/utils/styles.ets =====
// 公共 UI 样式工具：页面转场、动画参数等，供全局复用
// 用途：统一视觉体验，消除 14 个页面重复的 .transition() 代码
// 页面级转场：Apple 风格弹性滑入 + 渐显
// FastOutSlowIn 模拟 iOS 系统默认 ease-in-out - 先快后缓收，接近弹簧手感
// 用法：.transition(buildPageTransition())
export function buildPageTransition(): TransitionEffect {
  return TransitionEffect.OPACITY
    .animation({ duration: 360, curve: Curve.FastOutSlowIn })
    .combine(TransitionEffect.translate({ x: 24 })
      .animation({ duration: 380, curve: Curve.FastOutSlowIn }));
}

// ===== ./entry/src/main/ets/utils/tagIcons.ets =====
// 标签名 → 线条图标资源 映射 (Apple 风格极简线条 Logo)
// 每个标签独立图标；未匹配时回退到 tag_all (四角星)。
// 用 Map<string, Resource> + 字面量 $r(), 既规避 ArkTS 索引访问禁令,
// 又能在编译期校验 36 个资源确实存在。
const TAG_ICON_MAP: Map<string, Resource> = new Map<string, Resource>([
  // ===== 消费决策 =====
  ['数码选购', $r('app.media.tag_digital_shopping')],
  ['汽车买卖', $r('app.media.tag_car_trade')],
  ['家电评测', $r('app.media.tag_appliance_review')],
  ['外设推荐', $r('app.media.tag_peripheral')],
  ['男装搭配', $r('app.media.tag_menswear')],
  ['运动装备', $r('app.media.tag_sports_gear')],
  ['手机数码', $r('app.media.tag_phone_digital')],
  ['酒水品鉴', $r('app.media.tag_wine')],
  // ===== 动手实操 =====
  ['汽车养护', $r('app.media.tag_car_care')],
  ['数码维修', $r('app.media.tag_digital_repair')],
  ['家居维修', $r('app.media.tag_home_repair')],
  ['做饭教程', $r('app.media.tag_cooking')],
  ['健身动作', $r('app.media.tag_fitness')],
  ['电脑装机', $r('app.media.tag_pc_build')],
  ['摄影技巧', $r('app.media.tag_photography')],
  ['露营户外', $r('app.media.tag_camping')],
  // ===== 个人成长 =====
  ['职场沟通', $r('app.media.tag_workplace')],
  ['面试经验', $r('app.media.tag_interview')],
  ['搞钱心得', $r('app.media.tag_make_money')],
  ['理财规划', $r('app.media.tag_finance')],
  ['学习效率', $r('app.media.tag_study')],
  ['人际处世', $r('app.media.tag_social')],
  ['自我提升', $r('app.media.tag_self_improve')],
  // ===== 生活方式 =====
  ['旅行攻略', $r('app.media.tag_travel')],
  ['探店打卡', $r('app.media.tag_foodie')],
  ['游戏攻略', $r('app.media.tag_gaming')],
  ['影视推荐', $r('app.media.tag_movie')],
  ['书籍阅读', $r('app.media.tag_reading')],
  ['宠物日常', $r('app.media.tag_pet')],
  ['情感经营', $r('app.media.tag_relationship')],
]);
const TAG_ICON_ALL: Resource = $r('app.media.tag_all');
// 按标签名解析线条图标资源；未命中回退 tag_all
export function resolveTagIcon(name: string): Resource {
  const found: Resource | undefined = TAG_ICON_MAP.get(name);

```

```

    if (found !== undefined) {
        return found;
    }
    return TAG_ICON_ALL;
}

// ===== ./entry/src/main/ets/utils/theme.ets =====
// 主题工具：深色/浅色模式切换
// 策略（最保守版）：
// - setFollowSystem 不调 setColorMode, 只更新 AppStorage + preferences
// - toggleDarkMode 调 setColorMode(DARK/LIGHT) 强制指定
// - onConfigurationUpdate 只更新缓存, 不调 setColorMode
// - 消除所有 setColorMode 递归/异常路径
import { ConfigurationConstant, common } from '@kit.AbilityKit';
import { preferences } from '@kit.ArkData';
const PREF_NAME = 'bigbluebook_theme';
const KEY_FOLLOW_SYSTEM = 'followSystem';
const KEY_IS_DARK = 'isDarkMode';
let _appContext: common.ApplicationContext | null = null;
let _prefs: preferences.Preferences | null = null;
export function initThemeContext(ctx: common.ApplicationContext): void {
    _appContext = ctx;
    try {
        _prefs = preferences.getPreferencesSync(ctx, { name: PREF_NAME });
    } catch (e) {}
}

export function loadThemeFromPrefs(): void {
    if (_prefs === null) {
        return;
    }
    try {
        const follow: boolean = _prefs.getSync(KEY_FOLLOW_SYSTEM, true) as boolean;
        const dark: boolean = _prefs.getSync(KEY_IS_DARK, false) as boolean;
        AppStorage.setOrCreate('followSystem', follow);
        AppStorage.setOrCreate('isDarkMode', dark);
    } catch (e) {}
}

function saveFollowSystem(follow: boolean): void {
    if (_prefs === null) {
        return;
    }
    try {
        _prefs.putSync(KEY_FOLLOW_SYSTEM, follow);
        _prefs.flushSync();
    } catch (e) {}
}

function saveIsDark(dark: boolean): void {
    if (_prefs === null) {
        return;
    }
    try {
        _prefs.putSync(KEY_IS_DARK, dark);
        _prefs.flushSync();
    } catch (e) {}
}

export function getSystemDark(): boolean {
    return (AppStorage.get<boolean>('systemDark') ?? false) as boolean;
}

```

```

export function isDarkMode(): boolean {
    return (AppStorage.get<boolean>('isDarkMode') ?? false) as boolean;
}
export function isFollowSystem(): boolean {
    return (AppStorage.get<boolean>('followSystem') ?? true) as boolean;
}
export function toggleDarkMode(dark: boolean): void {
    AppStorage.setOrCreate('isDarkMode', dark);
    saveIsDark(dark);
    if (_appContext === null) {
        return;
    }
    try {
        _appContext.setColorMode(dark
            ? ConfigurationConstant.ColorMode.COLOR_MODE_DARK
            : ConfigurationConstant.ColorMode.COLOR_MODE_LIGHT);
    } catch (e) {
    }
}
export function setFollowSystem(follow: boolean): void {
    AppStorage.setOrCreate('followSystem', follow);
    saveFollowSystem(follow);
    // 不调 setColorMode - 避免任何副作用
    // 跟随系统时系统自动管颜色 (应用未调 setColorMode 即默认跟随)
    // 手动模式时由 toggleDarkMode 调 setColorMode 强制指定
    if (follow) {
        const sysDark: boolean = getSystemDark();
        AppStorage.setOrCreate('isDarkMode', sysDark);
    }
}

// ===== ./entry/src/main/ets/utils/toast.ets =====
import { UIContext, ShowToastOptions } from '@kit.ArkUI';
// 统一 toast 入口: 内部吞掉 showToast 可能抛出的异常, 避免各调用点触发 arkts-may-throw.
export function safeShowToast(uiContext: UIContext, options: ShowToastOptions):
void {
    try {
        uiContext.getPromptAction().showToast(options);
    } catch {
        // toast 失败不应阻断主流程, 静默兜底
    }
}

// ===== ./entry/src/ohosTest/ets/test/Ability.test.ets =====
import { hilog } from '@kit.PerformanceAnalysisKit';
import { describe, beforeAll, beforeEach, afterEach, afterAll, it, expect } from
 '@ohos/hypium';
export default function abilityTest() {
    describe('ActsAbilityTest', () => {
        // Defines a test suite. Two parameters are supported: test suite name and t
est suite function.
        beforeAll(() => {
            // Presets an action, which is performed only once before all test cases o
f the test suite start.
            // This API supports only one parameter: preset action function.
        })
        beforeEach(() => {
            // Presets an action, which is performed before each unit test case starts
            // The number of execution times is the same as the number of test cases d
efined by **it**.
            // This API supports only one parameter: preset action function.

```



```

    })
    afterEach(() => {
        // Presets a clear action, which is performed after each unit test case ends.
        // The number of execution times is the same as the number of test cases defined by **it**.
        // This API supports only one parameter: clear action function.
    })
    afterAll(() => {
        // Presets a clear action, which is performed after all test cases of the test suite end.
        // This API supports only one parameter: clear action function.
    })
    it('assertContain', 0, () => {
        // Defines a test case. This API supports three parameters: test case name, filter parameter, and test case function.
        hilog.info(0x0000, 'testTag', '%{public}s', 'it begin');
        let a = 'abc';
        let b = 'b';
        // Defines a variety of assertion methods, which are used to declare expected boolean conditions.
        expect(a).assertContain(b);
        expect(a).assertEqual(a);
    })
}

// ===== ./entry/src/ohosTest/ets/test/List.test.ets =====
import abilityTest from './Ability.test';
export default function testsuite() {
    abilityTest();
}

// ===== ./entry/src/test/List.test.ets =====
import localUnitTest from './LocalUnit.test';
export default function testsuite() {
    localUnitTest();
}

// ===== ./entry/src/test/LocalUnit.test.ets =====
import { describe, beforeAll, beforeEach, afterEach, afterAll, it, expect } from '@ohos/hypium';
export default function localUnitTest() {
    describe('localUnitTest', () => {
        // Defines a test suite. Two parameters are supported: test suite name and test suite function.
        beforeAll(() => {
            // Presets an action, which is performed only once before all test cases of the test suite start.
            // This API supports only one parameter: preset action function.
        });
        beforeEach(() => {
            // Presets an action, which is performed before each unit test case starts.
            // The number of execution times is the same as the number of test cases defined by **it**.
            // This API supports only one parameter: preset action function.
        });
        afterEach(() => {
            // Presets a clear action, which is performed after each unit test case ends.
            // The number of execution times is the same as the number of test cases d

```

```

efined by **it**.
    // This API supports only one parameter: clear action function.
  });
  afterAll(() => {
    // Presets a clear action, which is performed after all test cases of the
test suite end.
    // This API supports only one parameter: clear action function.
  });
  it('assertContain', 0, () => {
    // Defines a test case. This API supports three parameters: test case name
, filter parameter, and test case function.
    let a = 'abc';
    let b = 'b';
    // Defines a variety of assertion methods, which are used to declare expected boolean conditions.
    expect(a).assertContain(b);
    expect(a).assertEqual(a);
  });
});
}

// ===== ./hvisorfile.ts =====
import { appTasks } from '@ohos/hvisor-ohos-plugin';
export default {
  system: appTasks, /* Built-in plugin of Hvisor. It cannot be modified. */
  plugins: [] /* Custom plugin to extend the functionality of Hvisor. */
}

```